Literature review for Deep learning implementation on FPGA

Azadeh Hadadi, university UBFC, VIBOT Program, Le Creusot, France

Abstract this paper reviews the implementation of the deep learning network on an FPGA for both training and inference phases. The features such as FPGA resources used in an implementation, error of the prediction, speed up, throughput, power, and energy efficiency will be considered in this review. So far, different deep learning network have been proposed and accelerated on CPU cluster, GPUs, FPGA and ASIC chips; this paper focuses only on FPGA implementation. Nearly 25 papers, out of nearly hundred prepares have published in this regard since 2011 for deep learning acceleration on FPGA, were found with significant achievements and selected to present with more detail. The review mainly focuses on image processing application, although among the literatures some general-purpose solutions can be found due to their significant achievement. Majority of papers use the FPGA to implement only the inference stage and the training stage of the network, which includes extracting weights and biases of a network, is done outside FPGA by known software solution. The major objective the FPGA implementation was to reduce the latency, energy consumption and improve throughput. Although finding state-of-the-art literature depends on many factors and hard to define, it seems work presented in [21] has presented more clear achievements for image processing application.

***Index Terms*—Deep learning network, FPGA, inference, training**

I. Introduction

TO exploit the advances of FPGA implementation of the Convolutional Neural Network (CNN)/ Deep Neural Network (DNN), such as reducing the implementation time and fast evaluation and exploration, there has been extensive interest in FPGAs frameworks based on forward and backward data processing. However, there have been already other efforts to accelerate their implementation on other hardware platforms such as CPU clusters, GPUs and ASIC.

Despite the fact that the CNNs and DNNs have become particularly useful for acceleration of big data processing, there are huge demand to employ them as a part of embedded systems/subsystem in autonomous application such as robotics, mobile phones and automobiles, which require real-time processes with low latency and high accuracy. However, these algorithms are computationally very expensive because much time is spent on convolution operations [1].

The implementation of any kind of deep network is a difficult task, especially for embedded systems, because currently implementation on general purpose CPUs (even multiple core CPUS) are too slow and create huge amount of latency to process a single image, which motivates optimization strategies in terms of computational speed [2]. Other software-based CNNs use GPUs [3,4]. As GPUs consume a lot of energy and their development board usually needs a host motherboard, they are not suitable for all embedded systems [5]. Thus, FPGA based implementations as alternative solution has been proposed for low-power real-time embedded systems. An example of CNN/DNN accelerator is shown in Fig.1.

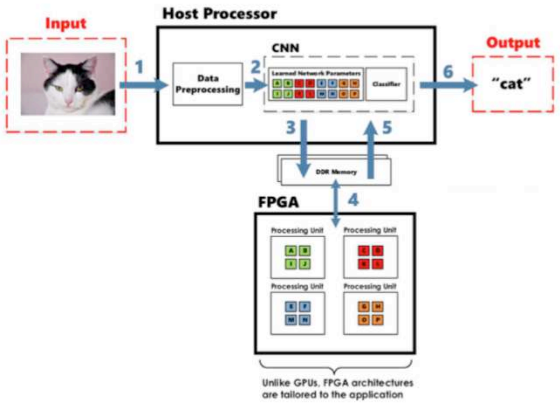

Fig.1: Example of typical FPGA-based implementation of Convolutional neural network acceleration on an FPGA (adopted from [6]).

All parts of the inference algorithm or part of it are implemented and accelerated on an FPGA, even some literature presented both training and inference implementation on an FPGA, which will be detailed in the following sections. To this end, the paper is organized as follows: section II details some selected implementations. Section III summarizes papers with interesting result. The paper will end up to conclusion and reference papers used in this survey.

This paper has been prepared for the partial fulfillment of the course
A. Hadadi is with the department of the engineering and computer science of the burgundy Franche-Comte University, university center Condorcet, 71200 Le Creusot, France (e-mail: a.hadadi1363@gmail.com).

II. Literature Review

Many papers have already been published since the deep learning was introduced to the literatures. Thousands of these papers addressed the software implementation of different framework, network architecture, database, training and validation for different applications. Only hundred of these papers addressed the hardware implementation and acceleration.

Wang et al. [7] presented a high performance implementation of deep learning neural networks on a Xilinx Zynq- zedboard xc7Z020, DUAL accelerator with three pipelined processing unit, using Xilinx Vivado by C code. The input and weights are divided and processed in parallel to improve the overall speedup. The floating-point coding was used in multiplication operations using 167 DSP blocks. Performance evaluation showed that an efficient Matrix Multiplication accelerator would be designed to improve the overall speed of the system when it takes more time. This technique is very useful for a large-scale deep network implementation. The Mnist and CIFAR-10 datasets were used for training, which was executed outside the FPGA in MATLAB. The implementation achieved up to 36.1× speedup for a 256 × 256 network size.

Chang et al. [8] presented a hardware implementation of a long-short term memory (LSTM) recurrent network on programmable logic Zynq 7020 FPGA. A two-layer LSTM neural network with a hidden layer of size 128 (weight matrix height) was adopted. The input and output of characters was a vector of size 65. The training of the network was codded in Torch7 and the FPGA implementation was done C++. Replicating the number of the networks running in parallel scaled the design. It used also four 32-bit DMA ports. Each port transmits two 16-bit streams given that the operations were done in 16 bits, while the bandwidth between FPGA and external host CPU full duplex DDR3 could reach 3.8 GB/s. The average error percentage for the output vector was 2.8%.

Xiao et al. [9] introduced a heterogeneous framework to explored acceleration of CNNs on FPGAs. The framework defines the CNN architecture in accordance with target FPGA resources such as block RAMs (BRAMs), DSPs and off-chip. The framework employs Xilinx Vivado HLS implementation tool and the Winograd's minimal filtering to provide an automated tool-chain to ease the map from a Caffe model to the FPGA bitstream generation. The authors could provide a comprehensive solution for mapping a wide variety of CNNs on FPGAs with the AlexNet and GGNet-E parameters (containing 16 convolutional layers, three fully connected layers, three max-pooling layers and one softmax layer), while the network training was carry out outside the FPGA device. The FPGA implementation targeted the Zynq ZC706 with 16-bit fixed-point data types. Given the VGG parameters for implementation, the platform could achieve on average 1.99× performance speedup compared to the work in [10] for various transfer constraints. The platform used a fusion strategy to decrease the feature- map transfer, which leads the energy savings of 68.2% on memory transfers for different transfer constraints. Given the Alexnet parameters, the proposed heterogeneous implementation improved performance by 99% on average, leading to additional 50% energy savings on computing resources. Compared to [10], this result offered an acceleration gain of 1.24×, just restricted by the exploration space of small solutions.

DiCecco et al. [11] presented a modified version of the CNN framework Caffe for FPGAs. This framework allowed specific FPGA implementations with the flexibility of reconfiguring the device whenever necessary, seamless memory transactions between the host CPU and the FPGA simple test benches, and ability to implement and execute multiple layers in parallel and to input two images simultaneously into the FPGA. Only the classification was done on-chip. The framework was validated on a Xilinx Virtex 7 XC7VX690T-2 FPGA using the Xilinx SDAccel development environment was used to implement a Winograd convolution engine. Some popular CNNs data sets such as AlexNet [12], GoogleNet [13], VGG [14], and Overfeat [15] were used to evaluate the framework with mix implementation of the CNN layers on the FPGA and a host CPU. The final CNN classifier model was tested on Soumith Chintala convent benchmarks experimental dataset [16]. The highest resource of 83.2% of the available LUTs on FPGA was utilized in post-placement and routing.

Ma et al. [17] presented an RTL compiler called ALAMO, which analyzed the algorithm structure and parameters. The automatically generated and integrate a set of modules to accelerate the algorithm on an FPGA taking into account the FPGA resources. Weights (approximately 250 MB) have to be stored in off-chip memory and transferred to FPGA during computation. A multiple word-length data (from 8 to 25-bits long for integer and fractional bits), dynamically adjustable, are used for the FPGA implementation. Only the inference was implemented on FPGA device and the training of the network was implemented in the software. The AlexNet and NiN [18] CNN was implemented on the Altera Stratix-V GXA7 FPGA using ALAMO compiler and 114.5 GOPS and 117.3 GOPS (with power consumption of 19.5W and 19.1W, initial power 16.5W) achieved, respectively. The FPGA implementation achieved a 1.9 speed-up comparing OpenCL version.

In [19], the authors presented a new flexible programming framework called CNNLab, which provided a high level programing language without involving a developer in implementation and synthesis phase of an FPGA. The training and inference phases were done on FPGA device. A CNN model defines with a se of parameters (convolution layer, normalization layer,...) in this framework. Several topologies were presented and implementations were tested on a GPU using CUDA and on an Intel-Altera DE5 FPGA with OpenCL. This work showed that the implementation on GPU is faster than FPGA (up to 1000x for FC layers) but the FPGA has better performance in terms of energy consumption due to its

limited hardware resources and lower work frequency.

Solazzo et al. [20] suggested a simplified implementation platform using C++ and Vivado HLS to speedup the design and implementation of CNNs on FPGAs allowing the designer to estimate resources needed for an implementation before synthesizing, average relative prediction error of 15% with a low variance (good result can be achieved for FPGAs with resources up flip-flops 1264 units). Splitting the input image and computing on different core in parallel can reduce the overall computation time and speedup significantly. It would also be possible to parallelize and speedup the overall computation by dividing the images to be processed among the different cores. Several parameters can be adjusted in this platform, i.e., the input image size, the number of convolutional layers (variable linear layers and fix classification layers, always 10 classes), the input and output feature maps, the kernel dimension. In this platform, the training is done outside FPGA and the weights are used in the implementation.

Fraser et al. [21] presented a new heterogeneous streaming framework, FINN. This new fast and flexible FPGA-based accelerators framework utilizes a set of optimizations for efficient mapping of Neural Networks. A fully connected, convolutional and pooling layer was implemented and tested on the SOC-FPGA Xilinx Zynq-7000 Series using Vivado C++ HLS version 2016.3, meaning only the prediction was done on FPGA device. The framework provided automated pipelining to meet the clock frequency target. Besides, the framework is able to assign network layers to achieve the user-defined throughput. Three neuronal architectures were adopted SFC (three layers with 256 neurons per layer), LFC (three layers with 1024 neurons per layer), VGG-16 (two convolution layers, one maxpool layer, with 64-128-256 input channels and two fully connected layers of 512 neurons in the output layer). The networks were trained outside the FPGA device using Tiny-CNN library using 1000 images selected of MNIST, CIFAR-10 and SVHN datasets. It seems that high classification accuracy could be achieved even when weights and activations were reduced from floating point to binary values, which means significant speedup in turn 66 TOPS (about 16x and 53x higher compared to 8-bit and 16-bit fixed-point operations, respectively). The results demonstrated that the difference in accuracy between low-precision and floating-point networks decreases as the network size increases. The proposed solution implemented on ZC706 FPGA utilizing 90% LUT resources, a 200MHz clock frequency and 1.6 GB/s of DRAM bandwidth. Prototypes exhibited good runtime efficiency, with ~80% for SFC and ~90% for LFC.

Fujii et al. [22] explored a neuron pruning technique, which eliminated part of weights based on Chainer framework [23], a sequential-input parallel-output with a fully connected layer and weight stored on the FPGA for fast memory access. The neuron pruning of fully connected layers was applied to the VGG-11 CNN on CIFAR-10 datasets. Only the prediction was implemented on the Xilinx Kintex-7 XC7K325T FPGA using C++ and Vivado 2016.2 IDE. The input of the networks was a normalized 32×32 image consists of 8-bit RGB color pixels. The number of neurons was reduced by 89.3% applying neuron pruning while maintaining 99% accuracy, which in turn means significant speedup. The FPGA implementation runs 219 and 12.5 times faster than CPU and GPU while achieves the energy efficiency of 125.28 and 17.88 times better than CPU and GPU respectively.

Dundar et al. [24] adopted an optimized streaming solution for a deep learning acceleration on an embedded hardware platform. The proposed solution was implemented on the Xilinx Kintex-7 XC7K325T FPGA for a video/image recognition application. The streaming transforms a high-level representation into operation codes to be executed by the hardware accelerator while using maximum available hardware resources. Different network architectures, with any number of filters and layers can be run. Wide variety of weight-level and node- level parallelization was presented. The validation of the method was done based on three channels and eight layers. Each single layer made of 3×128 convolution operations with 6×6 filters, a max-pooling operation with a 4×4 window size and a non-linearity operation. This network was trained for 20 classes, which was the size of network output, utilizing the Torch environment [25] on NVIDIA GPU. The FPGA implementation provided a performance of 247 Giga operations per second (GOPS) of throughput while consuming only 4 Watts of power.

Ortega et al. [26] introduced a layer-multiplexing scheme for the on-chip training of a deep neural network given the number of the hidden layer; the number of the neurons and training parameters are configurable. This scheme contained only one physical layer of neurons, but this layer could be reused iteratively in order to accelerate architectures with several hidden layers. The training phase was performed with 50 patterns and validated with 20 patterns, both patterns from the Iris plant dataset. The proposed scheme was implemented on the Virtex-5 XC5VLX110T FPGA using VHDL. The FPGA implementation presented 20-30 times speedup with respect to software version runs on CPU. This implementation degrades if the number of the hidden layers goes beyond 10.

Liang et al. [27] presented FP-BNN, a deep neural network for FPGAs, which drastically decreased hardware resource consumption while maintaining acceptable accuracy. The Resource-Aware Model Analysis (RAMA) was used to assess the resource cost, to help FP-BNN implementation on an FPGA. The RAMA help to removed the multipliers implementation bottleneck using bit-wise XNOR and shift operations (minimum DSP blocks were used) and optimized the access to parameters with data quantization and storing weighs on the FPGA. FP-BNN accelerator was implemented on Stratix-V FPGA for MNIST multi-layer perceptron, Cifar-10 ConvNet and AlexNet and evaluated. The training phase was carry out on

an IBM x3650 M4 server equipped with an NVIDIA Tesla K40.The inference of FP-BNN was faster than the previous design while energy efficiency was 10 times better over other computing platforms.

Chen et al. [28] employed the TVM compiler to adopt a deep learning high-level specification from other existing frameworks and generated an optimized low-level synthesizable logic for a diverse set of FPGAs applying many optimization features such as high-level operator fusion, mapping to arbitrary hardware primitives and memory latency hiding. It also provided an automated optimization of low-level programs to target hardware characteristics by employing a novel learning-based cost modeling method for the rapid exploration of code optimizations. The proposed solution implemented Vanilla Deep Learning Accelerator (VDLA), while using the previous accelerator proposal to reduce the architecture size on the FPGA. In this implementation huge network such as ResNet can be implemented on a low resource FPGA such as the one used in PYNG board. If the FPGA resource is tool small, a C runtime API library was developed to send the instruction and the weighting data to the target accelerator for execution. After that, the code generation algorithm converted an accelerator program to a series of API instruction and call. Offload VDLA convolution layers, a 16×16 matrix-vector unit clocked at 200MHz, was implemented to perform products of 8-bit values and accumulated them into a 32-bit register at each cycle, achieving 40× speedup.

Wang et al. [29] proposed hardware-software co-optimizer for the implementation of deep network with fully connected and convolutional layers taking into account hardware resources, and application requirements. The optimizer makes a trade off between accuracy and compression ratio adopting the general block-circular matrices. All operations were pipelined on the basic computing block. Both training and the inference phases of a small-to-medium-scale deep neural networks, using MNIST, SVHN and CIFAR-10 benchmark, were implemented on low-power FPGA Intel (Altera) Cyclone V 5CEA9 and higher-performance Xilinx Kintex-7 XC7K325T FPGA. The hardware strategy was to achieve ultra-high energy efficiency and performance for FPGA implementation by effective reconfiguration, batch processing, deep pipelining, resource reuse and hierarchical control. The accuracy degradations were constrained to be 1% to 2% between the original and block-circular matrix-based models. The proposed implementation achieved 152× speedup and 71×energy efficiency gain, compared with the IBM TrueNorth processor with the same accuracy. It achieved at least 31×energy efficiency gain, compared with the reference FPGA-based work [21].

Yazdabakhsh et al. [30] proposed the FlexiGAN end-to-end solution, which generated an optimized synthesizable FPGA accelerator from a high-level Generative Adversarial Network (GAN) specification. GANs architecture is an unsupervised classifier made of two nets, i.e., generative and discriminative. The proposed accelerator was implemented and synthesis for different variants of FlexiGAN architecture (only the inference), 3D-GAN and ArtGAN, on the Xilinx XCVU13P FPGA with 16-bit precision using Vivado Design Suite v2017.2. FlexiGAN was coupled with a novel template architecture that aimed at benefiting MIMD (Multiple Instruction, Multiple Data) and SIMD (Single Instruction, Multiple Data) execution models to harness ineffectual operations. The global and local instruction buffers were implemented with BRAMs and UltraRAMs. The proposed FlexiGAN-FPGA accelerator implementation, which operates at 190MHz, achieved a 2.2× higher speedup performance compared to an optimized conventional convolution design. The implementation in average is 2.6× (up to 3.7×) more energy efficient over Titan X GPU, while its clock rate was about 5.0× lower.

Noronha et al. [31] presented a new optimized implementation toolkit for training and inference of a deep learning network completely on an FPGA called Kibo. The toolkit developed in python, which provides a possibility to evaluate different parameters on the network implementation, speed up, accuracy and comparing functions commonly utilized in deep learning structures quickly and to customize the bit width of fixed-point weights and biases. Two networks, i.e., 1- Multi Layer Perceptron (MPL), 10bits fixed-point and 5 bits precision, and 2- recurrent neural network containing a Long-Short Term Memory (LSTM), were implemented on the Xilinx ZYNQ ZC706 FPGA to evaluate the toolkit for inference and training of time-series data. MLP implementation achieved 97.9% accuracy, which was comparable to state-of- the-art architectures using a single hidden layer. LSTM implementation showed that the loss quickly converge when using larger fixed-point representations. The LSTM implementation with a fixed-point, 16 bits with 16 bits precision, achieved the same accuracy as of floating-point precision.

Ding et al. [32] presented an efficient pipelined accelerator for Depth wise Separable CNN (DS-CNN) network on an FPGA where all layers are working simultaneously to improve the network throughput. The network was implemented an evaluated on the Intel ARRIA 10 FPGA using Intel Quartus Prime 16.0 for synthesis and ModelSim for simulation. The double-buffering-based memory channels were implemented to handle the dataflow between adjacent layers. The parallelization was exploited for the multiple input and output feature networks. Data type of 16 bits of fixed-point precision was employed for this implementation. For 2D convolution operators, the multiplication was implemented using parallel multipliers, and the adder tree summed all the results from multipliers in parallel in each clock cycle.

An experiment was setup to evaluate the performance of the proposed DS-CNN accelerator implementation. The accelerator was trained and tested on the MNIST and CIFAR-10 datasets. The results of experiments indicated

that the accelerator achieved 98.9 GOP/s and reached 17.6×speedup and 29.4×less power consumption comparing the CPU and GPU implementations, respectively.

III. Discussion and Comparison

Table I demonstrates different solution proposed for deep learning network implementation on an FPGA platform for training and inference as detail under section II.

Four features, i.e., FPGA resources consumed by implementation, speedup and throughput as well as energy efficiency achieved, were used to present each solution. Beside, FPGA implementation and the results achieved is usually application dependent. For instance, application such as image processing (ex. Image classification, object detection, character recognition and so on) consumes more FPGA resources and have high latency to calculate a deep learning network/map output comparing to signal or time series processing. Some solutions might provide quite fast, low latency, in turn high throughput, and energy efficient implementation for signal processing application while the same solution might not present the same achievement for image processing application. Therefore, the application is really matter.

TABLE I: Overview of selected implementations of deep learning network/maps on an FPGA

Ref.	FPGA platform	Feature studied in the literature				Application
		Resources	Speedup	Throughput	Energy	
[7]	Xilinx ZYNQ-zed		■			- MNIST - CIFAR-10
[8]	ZYNQ 7020			■		Character recognition
[9]	ZYNQ ZC706		■		■	General
[11]	Xilinx Virtex7	■				General
[17]	Altera Stratix-V			■		General
[20]	Various	■				Image classification
[21]	Xilinx ZYNQ-7000	■	■	■		Image classification
[22]	Xilinx Kintex7		■		■	CIFAR- 10
[24]	Xilinx Kintex-7			■		Image classification
[26]	Virtex-5		■			Iris dataset
[27]	Stratix-V		■		■	- MNIST - SVHN
[28]	PYNQ		■			General
[29]	Intel& Xilinx Kintex-7		■		■	- MNIST - SVHN - CIFAR- 10
[30]	Xilinx XCVU13P		■		■	General
[31]	Various		■			- MNIST -EEG signals
[32]	Intel Arria 10 FPGA		■	■		- MNIST - CIFAR- 10

Only [19][26][29][31] presented both training and inference of a deep learning network on an FPGA and all remaining literature used off-chip training methods and incorporated only the training result in the inference network. All solutions presented in table I have certain amount of flexibility in terms of size of the FPGA resources and adjusting the parameters to have better throughput or energy consumption and high speed up. Only [19] [24] and [17] studied the power change due to implementation parameter changes.

To make the long story short, it seems [21] has presented a better and clearer result about its implementation. They presented 80% and 90% of timing improvement for two proposed network.

IV. Conclusion

In a nutshell, the best way for deep learning implementation on FPGAs is to adopt the most appropriate architecture from the available architectures based on the target application, such as classification, segmentation, prediction, and so on. Always, it is preferable to run the training phase on a GPUs or CPU cluster, both to minimize the development time, and to overcome the large size that can be achieved by the best performing network. FPGA offers excellent acceleration support for the inference, allowing significant energy savings by lowering the working frequency and minimizing the prediction error. The available implementation tools such Vivado HLS simplify the implementation on FPGA by supporting high-level language like C or OpenCL (Vivado SDAccel) and synthesizable data-types. The use of frameworks accelerates the implementation time while the optimization-phase integration framework process is desirable. The precision can be adjusted by multiple word length in fixed point, and also with binary coding, which allows optimizing performances and use maximum out of the available FPGA resources. We can optimize the needed amount of resources (LUTs, flip-flops, etc.), the speedup, the energy, the throughput, and the prediction error rate at the same time by using optimizer. Among all papers it seem [21] presented a better and clearer result for the image processing application and it is selected as state-of-the-art paper.

References

[1] [32] C. Farabet, B. Martini, P. Akselrod, S. Talay, Y. LeCun, E. Culurciello, Hardware accelerated convolutional neural networks for synthetic vision systems, in: Proceed- ings of the IEEE International Symposium on Circuits and Systems (ISCAS), 2010, pp. 257–260. https://doi.org/10.1109/ISCAS.2010.5537908.

[2] [25] L.B. LeCun, Y. Bengio, P. Haffner, Gradient-based learning applied to doc- ument recognition, Proc. IEEE 86 (11) (1998) 2278–2324. https://doi.org/ 10.1109/5.726791.

[3] [12] A. Krizhevsky, One Weird Trick For Parallelizing Convolutional Neural Networks, 2014. in arXiv: 1404.5997.

[4] [13] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, E. Shel- hamer, cuDNN: Efficient Primitives For Deep Learning, 2014. in arXiv: 1410.0759.

[5] [9] G. Carneiro, J. Nascimento, P. Andrew, Bradley, deep learning models for classifying mammogram exams containing unregistered multi-view images and segmentation maps of lesions, book chapter

from Deep Learning for Medical Image Analysis, 2016. https://doi.org/10.1016/B978-0-12-810408-8.00019-5.
[6] [18] G. Lacey, G. Taylor, S. Areibi, Deep learning on FPGAs: past, present, and future, 2016. in arXiv: 1602.04283
[7] [24]C. Wang, Q. Yu, L. Gong, X. Li, Y. Xie, X. Zhou, DLAU: a scalable deep learning accelerator unit on FPGA, in: Proceedings of the IEEE Transactions on Computer- Aided Design of Integrated Circuits and Systems, Volume 36, Issue 3, March 2017 March, doi:101109/TCAD.2016.2587683.
[8] [27]A.X.M. Chang, B. Martini, E. Culurciello, Recurrent neural networks hardware implementation on FPGA, Int. J. Adv. Res. Electr. Electron. Instrum. Eng. 5 (1) (2016). January https://pdfs.semanticscholar.org/1a27/a30724ed0409791db99d96551d341807faed.pdf.
[9] [36]Q. Xiao, Y. Liang, L. Lu, S. Yan, Y.-W. Tai, Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs, in: Proceedings of the 54th Annual Design Automation Conference DAC '17, 2017, doi:10.1145/3061639.3062244.
[10] [70] M. Alwani, H. Chen, M. Ferdman, P. Milder, Fused-layer CNN accelerators, in: Pro- ceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitec- ture (MICRO), 2016, doi:10.1109/MICRO.2016.7783725.
[11] [40] R. DiCecco, G. Lacey, J. Vasiljevic, P. Chow, G. Taylor, S. Areibi, Caffeinated FPGAs: FPGA framework for convolutional neural networks, in: Proceedings of the Inter- national Conference on Field-Programmable Technology (FPT), 2016, pp. 7–9. Dec. 2016, Xi'an, China. doi: 10.1109/FPT.2016.7929549.
[12] [30]A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolu- tional neural networks, in: Proceedings of the Neural Information Processing Systems Conference, 2012, pp. 1097–1105. https://papers.nips.cc/paper/4824-imagenet- classification-with-deep-convolutional-neural-networks.pdf.
[13] [51] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S.E. Reed, D. Anguelov, D. Erhan, V. Van- houcke, A. Rabinovich, Going Deeper with Convolutions, 2014. in arXiv: 1409.4842.
[14] [52] K. Simonyan, A. Zisserman, in: Very deep convolutional networks for large scale visual recognition, 2015. in arXiv: 1409.1556.
[15] [53] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, Y. LeCun, OverFeat: Integrated Recognition, Localization and Detection Using Convolutional Networks, 2013. in arXiv: 1312.6229.
[16] [66] S. Chintala. Convnet-benchmarks. [Online]. Available: https://github.com/soumith/convnet-benchmarks
[17] [41] Y. Ma, N. Suda, Y. Cao, S. Vrudhula, J.S. Seo, ALAMO: FPGA acceleration of deep learning algorithms with a modularized RTL compiler, Integr. VLSI J. (2018), doi:10.1016/j.vlsi.2017.12.009.
[18] [74] M. Lin, Q. Chen, S. Yan, in: Network in network, 2014. arXiv: 1312.4400.
[19] [49] M. Zhu, L. Liu, C. Wang, Y. Xie, CNNLab: a novel parallel framework for neu- ral networks using GPU and FPGA – a practical study with trade-off analy- sis, 2016. Distributed, parallel, and cluster computing (cs.DC) https://arxiv.org/ pdf/1606.06234.pdf.
[20] [50] A. Solazzo, E.D. Sozzo, I.D. Rose, M.D. Silvestri, C Gianluca, Durelli, M.D Santambro- gio, Hardware design automation of convolutional neural networks, in: Proceedings of the IEEE Computer Society Annual Symposium on VLSI (ISVLSI), 2016, pp. 224– 229, doi:10.1109/isvlsi.2016.101.
[21] [54] Y. Umuroglu, N.J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, K. Vissers, FINN: A Framework for Fast, Scalable Binarized Neural Network Infer- ence, in: Proceedings of the 2017 ACM/SIGDA International Symposium on Field- Programmable Gate Arrays FPGA'17, Monterey, California, USA, 2017, pp. 65–74, doi:10.1145/3020078.3021744. February 22–24.
[22] [55] T. Fujii, S. Sato, H. Nakahara, M. Motomura, An FPGA realization of a deep convolutional neural network using a threshold neuron pruning, in: Proceed- ings of the ARC, in: LNCS, 10216, 2017, pp. 268–280, doi:10.1007/978-3-319- 56258-2_23.
[23] [71] Chainer: a powerful, flexible, and intuitive framework of neural networks. http://chainer.org/.
[24] [67] A. Dundar, J. Jin, B. Martini, E. Culurciello, Embedded streaming deep neural networks accelerator with applications, IEEE Trans. Neural Netw. Learn. Syst. 28 (7) (2016), doi:10.1109/TNNLS.2016.2545298.
[25] [68] R. Collobert, C. Farabet, K. Kavukcuoglu, Torch7: a matlab-like environment for machine learning, in: Proceedings of the BigLearn NIPS Work- shop, 2011 no. EPFL-CONF-192376 https://pdfs.semanticscholar.org/6074/c1108997e0c1f97dc3c199323a162ffe978d.pdf
[26] [69] F. Ortega-Zamorano, J.M. Jerez, I. Gómez, L. Franco, Deep neural network architec- ture implementation on FPGAS using a layer multiplexing scheme, in: proceedings of the 13th International Conference Distributed Computing and Artificial Intelligence, 2016, pp. 79–86. https://doi.org/:10.1007/978-3-319-40162-1_9.
[27] [72] S. Liang, S. Yin, L. Liu, W. Luk, S. Wei, FP-BNN: binarized neural network on FPGA, Neurocomputing (2017), doi:10.1016/j.neucom.2017.09.046.
[28] [73] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E.YanMeghan Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, A. Krishnamurthy, TVM: An automated end-to-end opti- mizing compiler for deep learning, 2018. 20 May https://arxiv.org/pdf/1802.04799 .pdf.
[29] [75] Y. Wang, C. Ding, Z. Li, G. Yuan, S. Liao, X. Ma, B. Yuan, X. Qian, J. Tang, Q. Qiu, X. Lin, Towards ultra-high performance and energy efficiency of deep learning systems: an algorithm-hardware co-optimization framework, 2018. 18 Feb https://arxiv.org/pdf/1802.06402.pdf
[30] [76] A. Yazdanbakhsh, M. Brzozowski, B. Khaleghi, S. Ghodrati, K. Samadi, N.S. Kim, H. Esmaeilzadeh, FlexiGAN: an end-to-end solution for FPGA acceleration of generative adversarial networks, in: Proceedings of the 26th IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM), 2018. https://www.cc.gatech.edu/~hadi/doc/paper/2018-fccm-flexigan.pdf.
[31] [77] D.H.Noronha, P.H.W.Leong, S.J.E.Wilton, Kibo: an open source fixed-point tool-kit for training and inference in FPGA-based deep learning networks, in: Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops, 2018 21–25 May, doi:10.1109/IPDPSW.2018.00034.
[32] [78] W. Ding, Z. Huang, Z. Huang, L. Tian, H. Wang, S. Feng, Designing efficient accelerator of depth wise separable convolutional neural network on FPGA, J. Syst. Archit. (2018) available online 28 December, doi:10.1016/j.sysarc.2018.12.008.